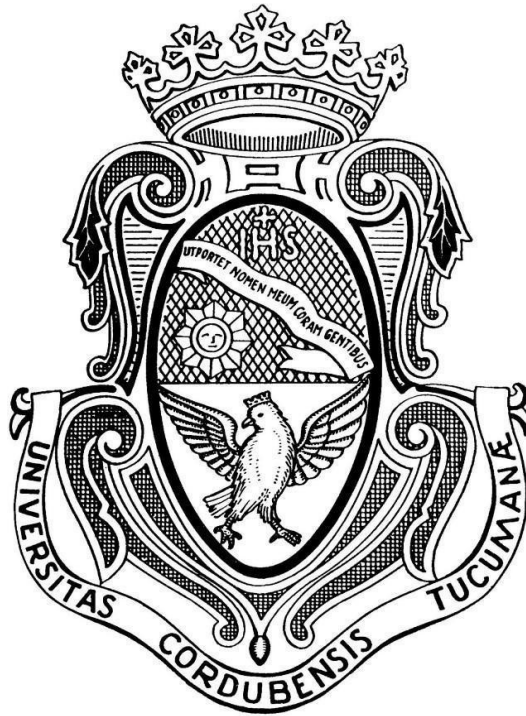


Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Sistemas de Computación

Trabajo Práctico N° 3 - Modo Protegido

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

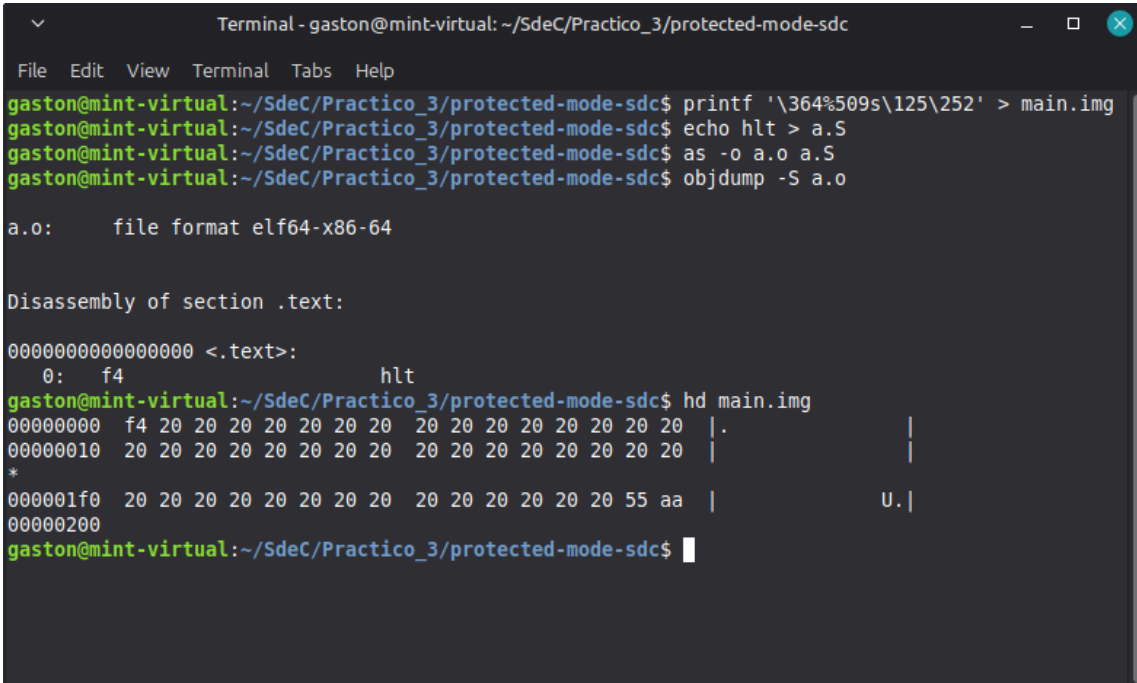
4 de mayo de 2026

Índice

1	Crear Imagen Booteable.....	2
2	UEFI y Coreboot	5
2.1	¿Qué es UEFI?¿Cómo puedo usarlo?	5
2.2	Bugs de UEFI	6
2.3	CSME y MEBx.....	6
2.4	Coreboot	6
3	Hello World	7
4	Linker.....	8
4.1	¿Qué es un Linker?¿Que hace?	8
4.2	¿Qué es la dirección que aparece en el script del Linker?¿Por qué es necesaria?	8
4.3	objdump vs hd	8
4.4	¿Para qué se utiliza -oformat binary en el linker?	10
5	Depuración de Ejecutables.....	10
6	Modo Protegido.....	12
6.1	Código en Assembler.....	12
6.2	Dos Descriptores de Memoria	13
6.3	Bits de Acceso.....	15
6.4	Registros de Segmento	15
7	Conclusiones	16
8	Anexo.....	16

1. Crear Imagen Booteable

Primero se ejecuta el comando “printf” para crear la imagen, como se ve en la siguiente figura:



```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$ printf '\364%509s\125\252' > main.img
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$ echo hlt > a.S
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$ as -o a.o a.S
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$ objdump -S a.o

a.o:          file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <.text>:
   0: f4                hlt
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$ hd main.img
00000000 f4 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |.
00000010 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |
*
000001f0 20 20 20 20 20 20 20 20 20 20 20 20 20 20 55 aa | U.
00000200
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc$
    
```

Figura 1: Creación de la imagen.

El 364 en octal equivale a f4 en hexadecimal, que corresponde a la instrucción “hlt” (halt), mientras que el 125 y el 252 corresponden a 55 y aa en hexadecimal, que forman la “firma” necesaria para que el archivo se reconozca como MBR (Master Boot Record). Por último, el 509 corresponde al relleno para que el archivo sea de 512 bytes, que es el tamaño necesario para este tipos de archivos.

A su vez, se utiliza el comando “echo” para guardar en un archivo de assembler la información que contiene la instrucción “hlt”. Luego se compila mediante el comando “as” (llama a GNU assembler) y se utiliza el comando “objdump” para comparar el código fuente con el desensamblado del archivo compilado, permitiendo verificar que el código hexadecimal que representa a la instrucción “hlt” es el x0f4.

Después, se utiliza el comando “hd” (hex dump), el cual muestra en la columna izquierda la posición hexadecimal dentro del archivo, en la columna del medio los bytes en hexadecimal (muestra 16 bytes por fila) y en la columna de la derecha la representación en ASCII (la cual está en su mayoría en blanco, porque los bytes de relleno “0x20” representan espacios en blanco). El asterisco indica que desde 00000010 hasta 000001f0 se repite la información (puros espacios en blanco).

Se confirma que al final del código aparece la “firma” de 55 y aa, por lo que la imagen funciona correctamente.

Para comprobar que la imagen funciona, se corre con el emulador “qemu”, como indica la Fig. 2. Se observa que funciona correctamente, y que simplemente se queda detenido por la instrucción “hlt”.

Se utiliza la aplicación “disks” para ver el nombre de los discos, como se ve en la Fig. 3. Acá se ve que el nombre del pendrive es sdb1.

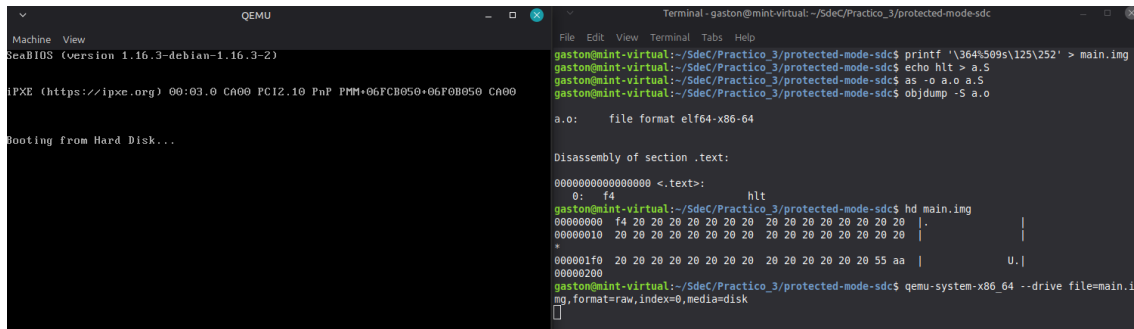


Figura 2: Simulación de la imagen “main.img” mediante el emulador “qemu”.

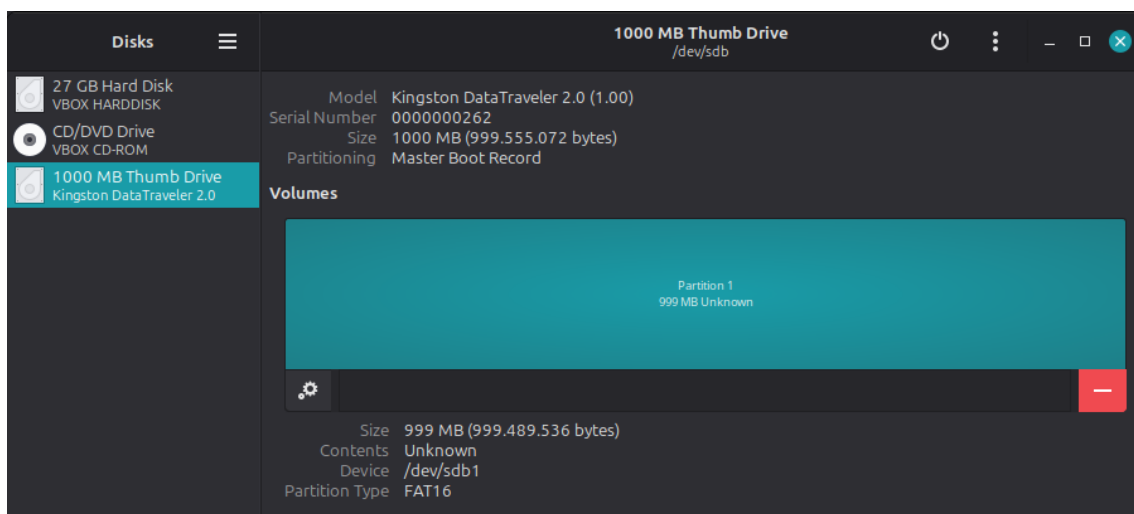


Figura 3: Nombre de discos mediante aplicación “disks”.

El nombre del disco del pendrive se utiliza para poder ejecutar el comando “dd”, como se ve en la Fig. 4.

Al intentar arrancar la pc mediante el pendrive, utilizando el menú de arranque como lo indica la Fig. 5, se observa que esta opción no está disponible. Esto se debe a que el equipo cuenta con un firmware UEFI Clase 3, el cual ha eliminado por completo el módulo de compatibilidad heredada (Compatibility Support Module - Legacy BIOS). Al carecer de este módulo, la placa base es incapaz de ejecutar código de 16 bits desde el MBR del pendrive y solo admite gestores de arranque modernos en formato .efi. Por ello, las pruebas a bajo nivel se realizarán exclusivamente mediante simulaciones en el emulador “qemu”.

2. UEFI y Coreboot

2.1. ¿Qué es UEFI? ¿Cómo puedo usarlo?

UEFI significa Unified Extensible Firmware Interface (Interfaz de Firmware Extensible Unificada). Se la puede pensar como un mini-sistema operativo que es el intermediario moderno entre el hardware físico y el verdadero sistema operativo.

Mientras que la BIOS antigua iniciaba en Modo Real de 16 bits (como el código MBR anterior) y solo podía direccionar 1 MB de memoria, UEFI arranca directamente en 32 o 64 bits, teniendo acceso a toda la memoria RAM desde el primer milisegundo.

Antes, la BIOS dependía de interrupciones de hardware programadas en ensamblador. UEFI, en cambio, provee APIs (Interfaces de Programación de Aplicaciones) escritas en lenguaje C, lo cual es útil porque abstrae del hardware. Al código en C no le importa si la pantalla está conectada por HDMI, DisplayPort o si es la pantalla integrada de la notebook, sino que basta con llamar a la función `OutputString` y el firmware de la UEFI se encarga del resto.

La BIOS solo leía sectores “crudos” de 512 bytes. UEFI lee sistemas de archivos FAT32 de forma nativa.

Si se quisiera programar un “Hello World” para UEFI y correrlo en una notebook actual, ya no se graban sectores crudos; en su lugar, se hace lo siguiente:

En lugar de escribir en ensamblador (.S), se escribe el código en lenguaje C y se compila a formato .efi, de forma que el código fuente ya no se convierte en un archivo .img crudo (binario plano), sino que se compila en un archivo ejecutable estandarizado llamado PE32+ (Portable Executable), y se le pone la extensión .efi.

Para hacer el arranque mediante pendrive, primero se lo formatea en FAT32, luego se crea una estructura de carpetas estricta: `\EFI \BOOT \`, y luego se pega el archivo .efi. Al encender la pc entrando al menú de arranque (mediante `esc` o una de las teclas `fx`) se puede seleccionar el arranque por el archivo del pendrive.

A continuación se presenta el código fuente en C para la impresión mediante UEFI:

```
#include <efi.h>
#include <efilib.h>

EFI_STATUS efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable) {
    // Se inicializa la libreria
    InitializeLib(ImageHandle, SystemTable);

    // Se llama a la funcion OutputString desde la tabla del sistema
    SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Hola Mundo en UEFI!\r\n");

    // Se devuelve el control al firmware
    return EFI_SUCCESS;
}
```

<efi.h> contiene los tipos de datos que se utilizan acá, como por ejemplo `EFI_STATUS`.

<efilib.h> contiene las funciones que se utilizan, por ejemplo `OutputString`.

En este caso la función principal no se llama solamente “main” sino que se llama “efi_main”, y devuelve un valor de tipo `EFI_STATUS`, que suele ser un número entero.

`ConOut` es Console Output, y desde allí se utiliza la función `OutputString`, que sirve para mostrar valores en pantalla.

2.2. Bugs de UEFI

1) BootHole (CVE-2020-10713)

Fue una vulnerabilidad masiva descubierta en 2020 que afectó al gestor de arranque GRUB2, el cual es utilizado por casi todas las distribuciones de Linux y algunos sistemas de recuperación de Windows. El problema residía en cómo GRUB2 leía su archivo de configuración de texto (grub.cfg). Un atacante podía modificar este archivo de texto metiendo parámetros extremadamente largos para provocar un desbordamiento de búfer (buffer overflow). Este bug permitió a los atacantes ejecutar código arbitrario dentro del entorno UEFI y evadir por completo el Secure Boot (Arranque Seguro). Como el archivo de configuración era solo texto, el Secure Boot no lo verificaba criptográficamente, a pesar de que el archivo ejecutable (.efi) sí estaba firmado legalmente.

2) BlackLotus (CVE-2022-21894 "Baton Drop")

Explotaba una vulnerabilidad conocida como "Baton Drop" en el gestor de arranque de Windows. El malware llevaba consigo versiones antiguas y vulnerables del gestor de arranque de Windows que aún poseían firmas digitales válidas de Microsoft. Al reemplazar los archivos modernos por estos antiguos, BlackLotus tomaba el control del sistema de forma "legal" a los ojos de la placa base, desactivando Windows Defender y el control de cuentas de usuario antes de que el usuario siquiera viera la pantalla de inicio de sesión.

2.3. CSME y MEBx

1) El Intel CSME (Converged Security and Management Engine) es un subsistema físico que viene integrado directamente en el chipset de casi todas las placas base Intel fabricadas desde el año 2008. No utiliza el procesador principal de la PC, sino que tiene su propio microprocesador dedicado (desde la versión 11 en adelante, utiliza una arquitectura Intel x86). Además, posee su propia memoria RAM dedicada (SRAM aislada) y su propio almacenamiento oculto en el chip de la BIOS.

El CSME tiene su propio Sistema Operativo: ejecuta una versión altamente modificada de MINIX 3 (un clon de Unix). Además, el CSME tiene privilegios por encima del sistema operativo e incluso de la UEFI. Puede leer y alterar toda la memoria RAM de la PC, encenderla, apagarla, y capturar gráficos, todo de forma invisible para el usuario.

Tiene conexión directa y exclusiva a la tarjeta de red (Ethernet o Wi-Fi). Parte del tráfico de internet se desvía físicamente hacia el CSME antes de llegar al sistema operativo.

Intel lo diseñó para centralizar la seguridad criptográfica y permitir la gestión remota a nivel corporativo.

2) El MEBx es una "interfaz de usuario" que permite comunicarse con el CSME antes de que cargue el sistema operativo normal. Es un módulo de firmware opcional que Intel le proporciona a fabricantes como ASUS, Dell o HP para que lo incrusten dentro de su UEFI/BIOS.

Su propósito principal: La función fundamental de la MEBx es realizar el "aprovisionamiento" (la configuración inicial de red y seguridad) del módulo Intel AMT para que el departamento de TI pueda tomar el control remoto.

2.4. Coreboot

Coreboot (anteriormente conocido como LinuxBIOS) es un proyecto de software de código abierto diseñado para reemplazar el firmware propietario (la BIOS o UEFI) que viene de fábrica en las placas base de las computadoras. Su filosofía de diseño es radicalmente opuesta a la de UEFI. Mientras que UEFI intenta ser un "mini sistema operativo" con controladores de red, interfaces gráficas y analizadores de imágenes, coreboot sigue un principio minimalista: hacer lo mínimo indispensable para inicializar el hardware (RAM, CPU, chipset).

Una vez que coreboot inicializa el hardware, le pasa el control a lo que se llama un Payload (Carga útil), como por ejemplo:

SeaBIOS: Si se quiere emular una vieja BIOS Legacy (MBR).

TianoCore / EDK II: Si se quiere tener una UEFI moderna y arrancar Windows 11.

GRUB / Linux Kernel: Se puede hacer que coreboot cargue directamente el núcleo de Linux desde el chip de la placa base, saltando por completo el gestor de arranque del disco duro.

¿Qué productos lo incorporan?

Chromebooks de Google: Este es el caso de éxito más grande. Todos los dispositivos con ChromeOS (fabricados por HP, Acer, Lenovo, ASUS, etc.) utilizan coreboot como su firmware base de fábrica. Google lo exige por su velocidad y seguridad.

System76: Una empresa estadounidense muy respetada que fabrica computadoras portátiles y de escritorio optimizadas para Linux (creadores de Pop!_OS). Han hecho una transición masiva para enviar sus equipos con coreboot preinstalado.

Equipos de red y servidores: Muchas empresas de infraestructura y centros de datos (incluyendo a Facebook/Meta y sus diseños de Open Compute Project) utilizan coreboot en sus servidores para eliminar dependencias de fabricantes propietarios.

¿Cuáles son las ventajas de su utilización?

Velocidad de arranque extrema: Al eliminar todos los chequeos innecesarios, interfaces gráficas pesadas y protocolos que UEFI carga por defecto, coreboot puede inicializar el hardware en milisegundos. Un Chromebook con coreboot suele encender y estar en la pantalla de inicio de sesión en menos de 5 segundos.

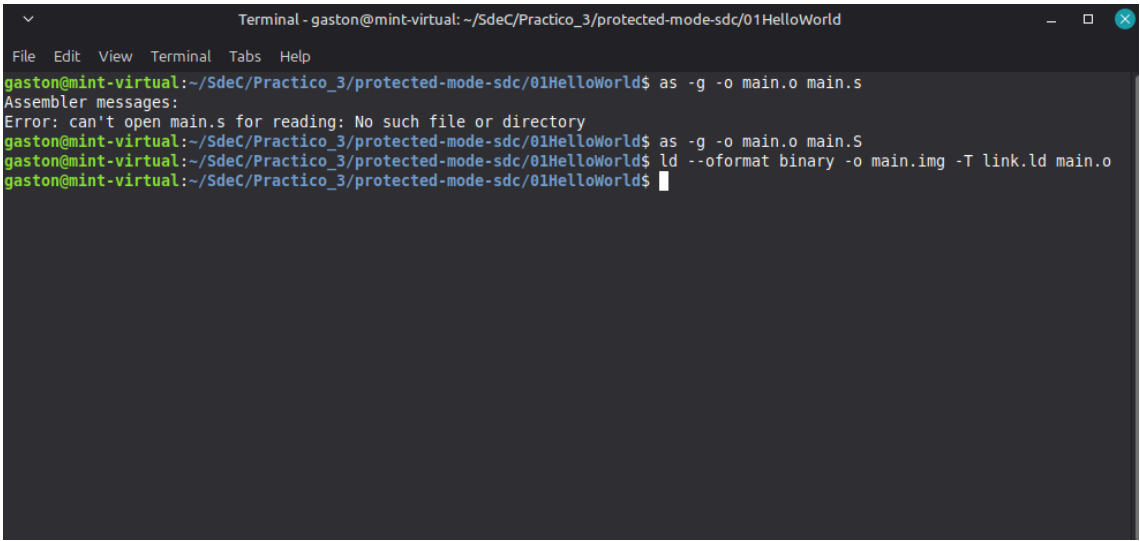
Seguridad y Auditoría (Código Abierto): A diferencia de las UEFI propietarias (donde no se puede ver el código fuente y hay que confiar a ciegas en el fabricante), el código de coreboot es público. Cualquier investigador de seguridad puede auditarlo.

Neutralización del Intel ME (CSME): En el mundo de coreboot, existe una herramienta muy famosa llamada me_cleaner. Como se tiene el control total del firmware al compilar coreboot, se puede usar esta herramienta para borrar casi el 90 % del sistema operativo secreto de Intel, dejando solo la parte vital para que la PC encienda, eliminando así los riesgos de espionaje o control remoto.

Ausencia de “Vendor Lock-in” (Dependencia del fabricante): Si ASUS o HP deciden dejar de lanzar actualizaciones de seguridad para la UEFI de una placa base después de 3 años, entonces queda vulnerable. Si la placa está soportada por coreboot, la comunidad mantiene las actualizaciones de seguridad indefinidamente.

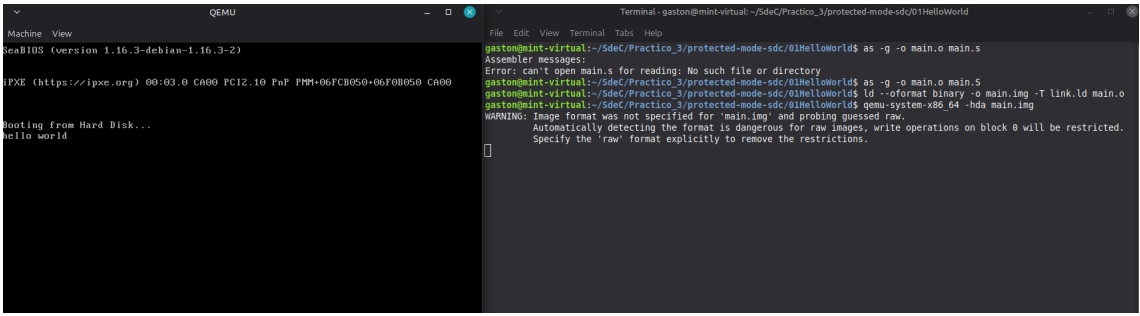
3. Hello World

Como indica la figura 6 primero se busca compilar el archivo main.S para obtener un archivo objeto, es decir main.o. Luego se utiliza el linker para crear la imagen, llamada main.img, y se simula mediante qemu, como se muestra en la figura 7. La simulación es exitosa, ya que se muestra en pantalla un “hello world”.



```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ as -g -o main.o main.s
Assembler messages:
Error: can't open main.s for reading: No such file or directory
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ as -g -o main.o main.S
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ ld --oformat binary -o main.img -T link.ld main.o
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$
```

Figura 6: Compilación y Linkeo.



```
QEMU
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
IPXE (https://ipxe.org) 00:03.0 CA00 FC12.10 FnP PMM-06FCB050+06F0B050 CA00
Booting from Hard Disk...
hello world

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ as -g -o main.o main.s
Assembler messages:
Error: can't open main.s for reading: No such file or directory
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ as -g -o main.o main.S
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ ld --oformat binary -o main.img -T link.ld main.o
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ qemu-system-x86_64 -hda main.img
WARNING: Image format was not specified for 'main.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$
```

Figura 7: Simulación de main.img mediante qemu.

4. Linker

4.1. ¿Qué es un Linker? ¿Que hace?

Un linker es un programa del sistema cuya única función es tomar uno o más archivos objeto (pedazos de código ya traducidos a lenguaje máquina por el compilador, pero que están incompletos) y combinarlos para formar un único archivo ejecutable.

El trabajo del linker se divide en dos grandes fases críticas:

1) Resolución de Símbolos:

Cuando se escribe código y se llama a una función que escribió alguien más, el compilador de C no sabe cómo funciona esa función, por lo que el Linker revisa todos los archivos objeto que se generaron y todas las librerías que se le indicaron, busca dónde está guardado realmente el código de las funciones y “conecta” la llamada que se hace con el código real de la función. Si se llama a una función que no existe o no se incluye la biblioteca, el linker lanzará un error de referencia indefinida.

2) Relocalización:

Cuando el compilador traduce un archivo .c a un archivo objeto (.o), asume que el programa será el único en el mundo y que empezará en la dirección de memoria 0, pero un programa real está compuesto por decenas de archivos objeto. Entonces el Linker agarra todos los pedazos de código de los distintos archivos objeto y los apila uno detrás de otro. El linker recalcula matemáticamente todas las direcciones de memoria de las variables y funciones para que apunten a su nueva ubicación definitiva dentro del archivo ejecutable.

4.2. ¿Qué es la dirección que aparece en el script del Linker? ¿Por qué es necesaria?

En el script del linker, esa dirección se conoce formalmente como la VMA (Virtual Memory Address). En la sintaxis de los scripts del linker de GNU, generalmente se define usando un punto (.), que se llama Contador de Posición.

Cuando en un script del linker se escribe algo como “. = 0x7C00;”, se le está avisando al linker que a partir de esta línea, todo el código y las variables que se van a poner a continuación van a vivir físicamente en la dirección de memoria RAM 0x7C00 cuando la computadora encienda. Entonces es necesario cargar esta dirección para que se puedan referenciar correctamente las instrucciones del código que hay que ejecutar.

4.3. objdump vs hd

Acá se realiza la comparación entre los comandos objdump y hd sobre los archivos correspondientes a “hello world”. Entonces, primero se ejecuta el objdump sobre el archivo main.o, según la figura 8, y luego se ejecuta el hd sobre main.img, según la figura 9.

En el objdump, se ve que las líneas de código arrancan en la posición 0, y también se observa que el mensaje msg está en la posición f. El número “be” equivale a la instrucción mov, donde se copia un valor inmediato de 16 bits al registro “si”. Como aún no se sabe el valor a mover, se ponen ceros (00 00).

Por otro lado, en el hd se observa que las líneas de código también arrancan en la posición 0, y al principio del código se observa “be 0f 7c”, por lo que se cargó la dirección donde está el mensaje. Cabe aclarar que “7c 00” era donde comienza el código, entonces se le suma “f” (y además se usa Little Endian).

Además, se observa que el código contiene relleno para completar los 512 bytes, y que contiene la “firma” de 55 aa al último.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ objdump -S main.o

main.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <loop-0x5>:
.code16
    mov $msg, %si
    0:  be 00 00 b4 0e          mov     $0xeb40000,%esi

0000000000000005 <loop>:
    mov $0x0e, %ah
loop:
    lodsb
    5:  ac                    lods    %ds:(%rsi),%al
    or %al, %al
    6:  08 c0                or     %al,%al
    jz halt
    8:  74 04                je     e <halt>
    int $0x10
    a:  cd 10                int    $0x10
    jmp loop
    c:  eb f7                jmp    5 <loop>

000000000000000e <halt>:
halt:
    hlt
    e:  f4                hlt

000000000000000f <msg>:
    f:  68 65 6c 6c 6f      push   $0x6f6c6c65
    14:  20 77 6f              and    %dh,0x6f(%rdi)
    17:  72 6c                jnb    85 <msg+0x76>
    19:  64                  fs
    ...
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$

```

Figura 8: objdump sobre main.o.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$ hd main.img
00000000  be 0f 7c b4 0e ac 08 c0 74 04 cd 10 eb f7 f4 68  |...t....h|
00000010  65 6c 6c 6f 20 77 6f 72 6c 64 00 66 2e 0f 1f 84  |ello world.f...|
00000020  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f|
00000030  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f..|
00000040  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f..|
00000050  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...|
00000060  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84  |.f.....f....|
00000070  00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f...f|
00000080  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f...f..|
00000090  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f...f...|
000000a0  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...f....|
000000b0  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84  |.f.....f...f....f|
000000c0  00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f...f....f..|
000000d0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f...f....f...|
000000e0  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f...f....f...f..|
000000f0  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...f....f...f...|
00000100  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84  |.f.....f...f....f...f...f..|
00000110  00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f...f....f...f...f...|
00000120  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f...f....f...f...f...f..|
00000130  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f...f....f...f...f...f...|
00000140  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...f....f...f...f...f...f..|
00000150  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84  |.f.....f...f....f...f...f...f...f...f..|
00000160  00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f...f....f...f...f...f...f...f...f..|
00000170  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f...f....f...f...f...f...f...f...f...f..|
00000180  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f...f....f...f...f...f...f...f...f...f...f..|
00000190  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...f....f...f...f...f...f...f...f...f...f...f..|
000001a0  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84  |.f.....f...f....f...f...f...f...f...f...f...f...f...f...f..|
000001b0  00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66  |...f.....f...f....f...f...f...f...f...f...f...f...f...f...f..|
000001c0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00  |...f.....f...f....f...f...f...f...f...f...f...f...f...f...f...f..|
000001d0  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f  |...f.....f...f....f...f...f...f...f...f...f...f...f...f...f...f...f..|
000001e0  1f 84 00 00 00 00 66 2e 0f 1f 84 00 00 00 00  |...f.....f...f....f...f...f...f...f...f...f...f...f...f...f...f...f...f..|
000001f0  00 66 2e 0f 1f 84 00 00 00 00 0f 1f 00 55 aa  |.f.....U.|
00000200
gaston@mint-virtual:~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld$

```

Figura 9: hd sobre main.img.

4.4. ¿Para qué se utiliza `-oformat binary` en el linker?

En el comando

```
ld --oformat binary -o main.img -T link.ld main.o
```

se utiliza la opción `-oformat binary`.

Normalmente, el linker crea archivos con formatos complejos para sistemas operativos (como ELF en Linux o PE en Windows) que tienen encabezados y metadatos. Este parámetro le exige al linker que no cree esas cosas y genere un binario plano y crudo (raw binary). La BIOS de una PC solo entiende binarios crudos de exactamente 512 bytes al arrancar.

5. Depuración de Ejecutables

Primero se lanza `qemu` y se le da el control a `gdb` mediante los comandos que aparecen en la figura 10. Luego se introducen los breakpoints en la dirección de arranque (0x7C00), antes y después de la instrucción 0x10 (0x7C0A y 0x7C0C según lo visto en `objdump`), como se ve en la figura 11

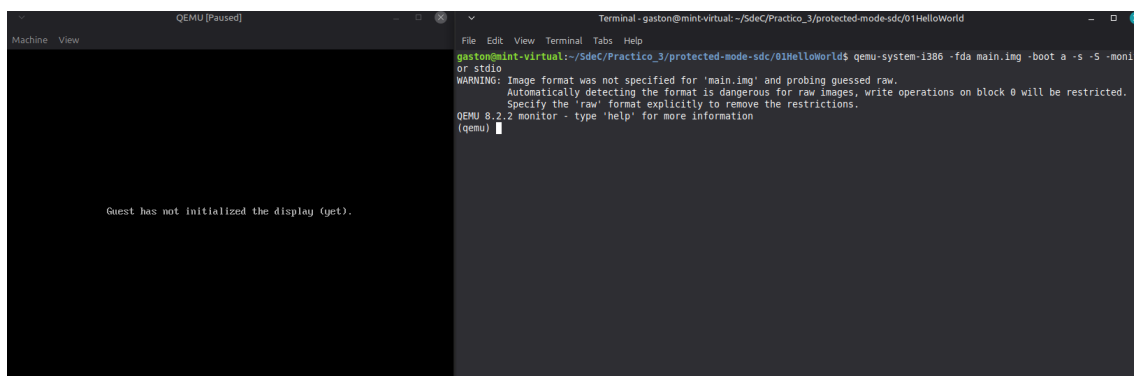


Figura 10: qemu con gdb.

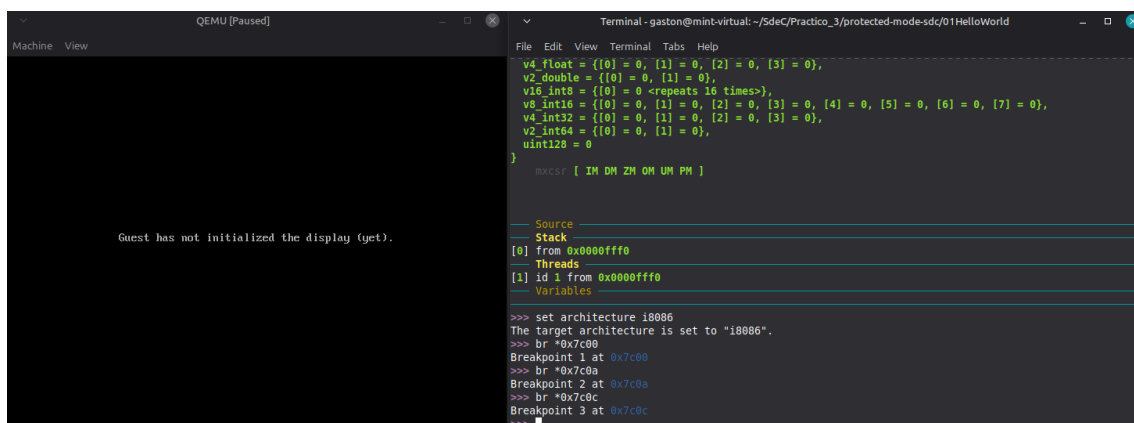
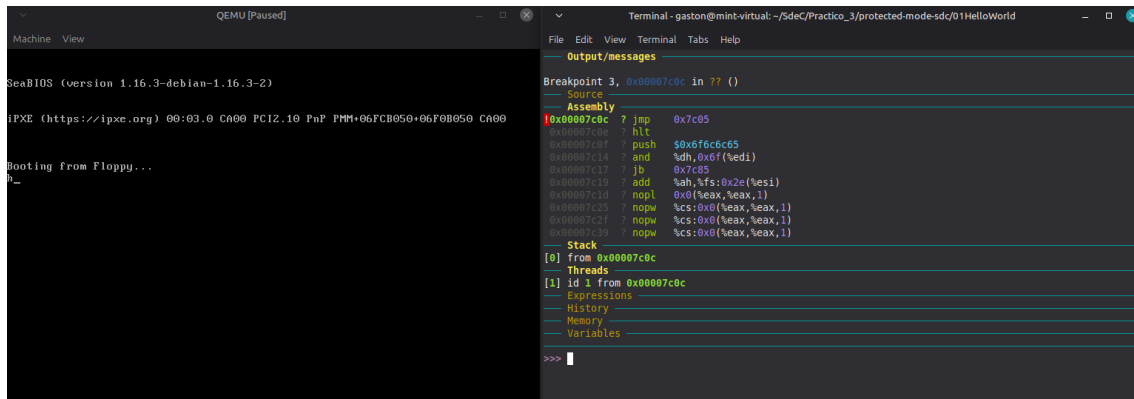


Figura 11: Breakpoints en 0x7C00, 0x7C0A y 0x7C0C.

Ahora se puede ejecutar el comando `continue` para imprimir los caracteres de “hello world” uno por uno, como muestran las figuras 12, 13 y 14.



QEMU (Paused) Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld

Machine View

SeaBIOS (version 1.16.3-debian-1.16.3-2)

IPXE (https://ipxe.org) 00:03.0 CA00 PC12.10 FnP PMM+06FCB050+06F0B050 CA00

Booting from Floppy...

0_

Output/messages

Breakpoint 3, 0x00007c0c in ?? ()

Source

Assembly

```

0x00007c0c 7 jmp 0x7c05
0x00007c0e 7 hlt $0x6f6c6c65
0x00007c0f 7 push $0x6f6c6c65
0x00007c14 7 and %dh,0x6f(%edi)
0x00007c17 7 jb 0x7c05
0x00007c19 7 add %ah,%fs:0x2e(%esi)
0x00007c1d 7 nopl 0x0(%eax,%eax,1)
0x00007c25 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c2f 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c39 7 nopw %cs:0x0(%eax,%eax,1)

```

Stack

[0] from 0x00007c0c

Threads

[1] id 1 from 0x00007c0c

Expressions

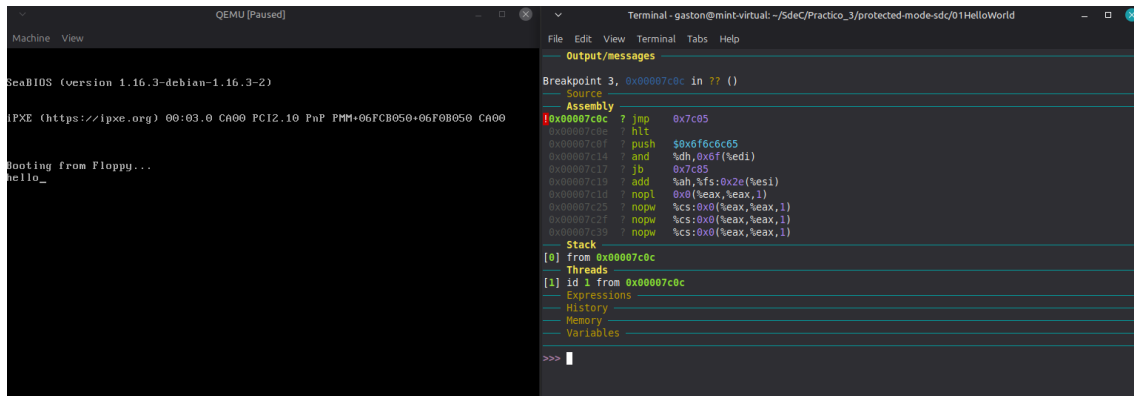
History

Memory

Variables

>>>

Figura 12: Impresión en pantalla 1.



QEMU (Paused) Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld

Machine View

SeaBIOS (version 1.16.3-debian-1.16.3-2)

IPXE (https://ipxe.org) 00:03.0 CA00 PC12.10 FnP PMM+06FCB050+06F0B050 CA00

Booting from Floppy...

hello_

Output/messages

Breakpoint 3, 0x00007c0c in ?? ()

Source

Assembly

```

0x00007c0c 7 jmp 0x7c05
0x00007c0e 7 hlt $0x6f6c6c65
0x00007c0f 7 push $0x6f6c6c65
0x00007c14 7 and %dh,0x6f(%edi)
0x00007c17 7 jb 0x7c05
0x00007c19 7 add %ah,%fs:0x2e(%esi)
0x00007c1d 7 nopl 0x0(%eax,%eax,1)
0x00007c25 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c2f 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c39 7 nopw %cs:0x0(%eax,%eax,1)

```

Stack

[0] from 0x00007c0c

Threads

[1] id 1 from 0x00007c0c

Expressions

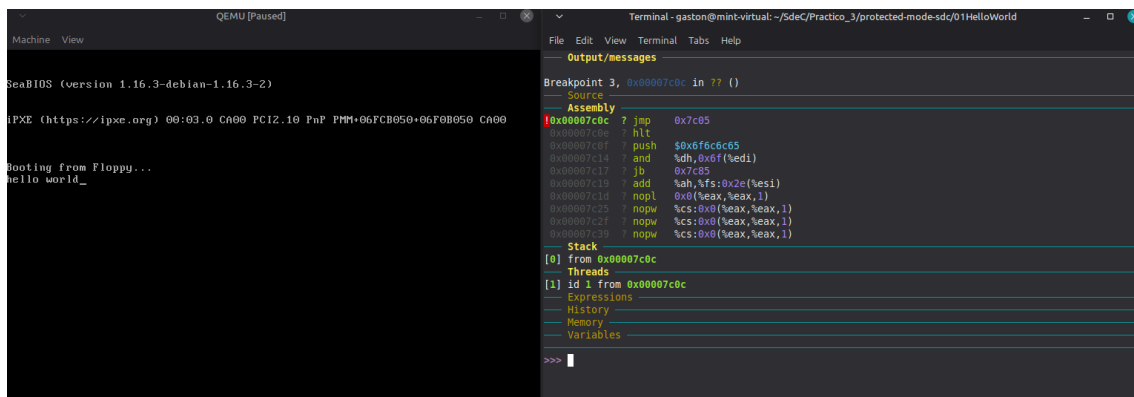
History

Memory

Variables

>>>

Figura 13: Impresión en pantalla 2.



QEMU (Paused) Terminal - gaston@mint-virtual: ~/SdeC/Practico_3/protected-mode-sdc/01HelloWorld

Machine View

SeaBIOS (version 1.16.3-debian-1.16.3-2)

IPXE (https://ipxe.org) 00:03.0 CA00 PC12.10 FnP PMM+06FCB050+06F0B050 CA00

Booting from Floppy...

hello world_

Output/messages

Breakpoint 3, 0x00007c0c in ?? ()

Source

Assembly

```

0x00007c0c 7 jmp 0x7c05
0x00007c0e 7 hlt $0x6f6c6c65
0x00007c0f 7 push $0x6f6c6c65
0x00007c14 7 and %dh,0x6f(%edi)
0x00007c17 7 jb 0x7c05
0x00007c19 7 add %ah,%fs:0x2e(%esi)
0x00007c1d 7 nopl 0x0(%eax,%eax,1)
0x00007c25 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c2f 7 nopw %cs:0x0(%eax,%eax,1)
0x00007c39 7 nopw %cs:0x0(%eax,%eax,1)

```

Stack

[0] from 0x00007c0c

Threads

[1] id 1 from 0x00007c0c

Expressions

History

Memory

Variables

>>>

Figura 14: Impresión en pantalla 3.

Luego de imprimir los caracteres, el programa se detiene.

6. Modo Protegido

6.1. Código en Assembler

El objetivo es crear un código en assembler que pase a modo protegido sin utilizar macros.

```
.code16                # El procesador arranca en Modo Real de 16 bits
.global _start

_start:
    # 1. Apagar las interrupciones
    cli

    # 2. Cargar la Tabla Global de Descriptores (GDT)
    lgdt gdt_descriptor

    # 3. Activar el bit de Modo Protegido (PE) en el registro de control cr0
    mov %cr0, %eax
    or $1, %eax        # Pone a 1 el bit menos significativo
    mov %eax, %cr0

    # 4. Salto Lejano (Far Jump) para limpiar el pipeline y entrar a 32 bits
    # 0x08 es el "Selector de Segmento" que apunta al Segmento de Código en nuestra GDT
    ljmp $0x08, $modo_protegido

# -----
.code32                # A partir de acá, se tiene un sistema moderno de 32 bits
modo_protegido:
    # 5. Configurar los registros de segmento de datos
    # 0x10 es el "Selector de Segmento" que apunta al Segmento de Datos en la GDT
    mov $0x10, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %fs
    mov %ax, %gs
    mov %ax, %ss

    # 6. Imprimir una 'X' en la pantalla escribiendo directamente en la memoria de video VGA
    # (Ya no se puede usar la interrupción de la BIOS int 0x10)
    mov $0x2F58, %ax    # 58 es 'X' en ASCII. 2F es el color (F=Blanco, 2=Fondo verde)
    mov %ax, 0xB8000    # 0xB8000 es la dirección física en RAM de la memoria de video

    # 7. Bucle infinito para no ejecutar basura
bucle_infinito:
    hlt
    jmp bucle_infinito

# -----
# TABLA GLOBAL DE DESCRIPTORES (GDT)
# -----
.align 8
gdt_start:

# Primer descriptor: Obligatoria Nulo (Requisito de Intel)
gdt_null:
    .quad 0x0000000000000000
```

```
# Segundo descriptor: Segmento de Código (Offset = 0x08 bytes desde el inicio)
gdt_code:
    .word 0xffff          # Límite (bits 0-15)
    .word 0x0000          # Base (bits 0-15)
    .byte 0x00            # Base (bits 16-23)
    .byte 0b10011010      # Byte de Acceso: Presente, Anillo 0, Es Código, Ejecutable/Legible
    .byte 0b11001111      # Flags (Granularidad 4K, 32 bits) y Límite (bits 16-19)
    .byte 0x00            # Base (bits 24-31)

# Tercer descriptor: Segmento de Datos (Offset = 0x10 bytes desde el inicio)
gdt_data:
    .word 0xffff          # Límite (bits 0-15)
    .word 0x0000          # Base (bits 0-15)
    .byte 0x00            # Base (bits 16-23)
    .byte 0b10010010      # Byte de Acceso: Presente, Anillo 0, Es Datos, Legible/Escribible
    .byte 0b11001111      # Flags (Granularidad 4K, 32 bits) y Límite (bits 16-19)
    .byte 0x00            # Base (bits 24-31)

gdt_end:

# Puntero a la GDT que requiere la instrucción 'lgdt'
gdt_descriptor:
    .word gdt_end - gdt_start - 1  # Tamaño de la tabla menos 1
    .long gdt_start                 # Dirección de inicio de la tabla

# -----
# Firma del Sector de Arranque (Magic Number)
# -----
.fill 510 - (. - _start), 1, 0      # Rellena con ceros hasta el byte 510
.word 0xAA55                        # Bytes 511 y 512
```

6.2. Dos Descriptores de Memoria

Diferencia 1: La escritura en memoria (El Offset)

La forma en la que se le dice al procesador que dibuje la letra cambia radicalmente porque el punto de referencia (el cero) se ha movido.

Modelo Plano (Memoria unificada)

```
mov $0x2F58, %ax
```

Se escribe en la dirección física real (0xB8000) porque la base del segmento de datos es 0. Matemáticamente: $0 + 0xB8000 = 0xB8000$.

Modelo Segmentado (Datos separados)

```
mov %ax, 0xB8000
movw $0x2F58, 0x0000
```

Se escribe en la dirección "cero" del segmento de datos. Como la base del segmento ya está configurada en 0xB8000, la matemática interna hace: $0xB8000 + 0 = 0xB8000$.

Diferencia 2: El Descriptor de Datos (gdt_data)

Esta es la parte del script donde se definen las reglas. Se cambia la "Dirección Base" (dónde empieza el segmento) y los "Flags" (el tamaño del segmento).

El código del Modelo Plano (Base 0, Tamaño 4GB):

Fragmento de código

```
gdt_data:
    .word 0xffff      # Límite
    .word 0x0000      # Base 0-15: 0x0000
    .byte 0x00        # Base 16-23: 0x00
    .byte 0b10010010
    .byte 0b11001111  # Flags: Granularidad 4KB (Bit superior en 1)
    .byte 0x00        # Base 24-31: 0x00
```

El código del Modelo Segmentado (Base 0x000B8000, Tamaño 64KB):
Fragmento de código

```
gdt_data:
    .word 0xffff      # Límite
    .word 0x8000      # Base 0-15: 0x8000
    .byte 0x0B        # Base 16-23: 0x0B
    .byte 0b10010010
    .byte 0b01000000  # Flags: Granularidad 1 Byte (Bit superior en 0)
    .byte 0x00        # Base 24-31: 0x00
```

El detalle de los cambios:

Fragmentación de la Base (0x0B8000):

En el modelo plano, todos los bytes de la Base eran 0x00. En el segmentado, se construye el número de 32 bits de la dirección de video (que se escribe completo como 0x000B8000) cortándolo en pedazos:

Bits 0-15 toman los últimos cuatro caracteres: 0x8000.

Bits 16-23 toman los siguientes dos caracteres: 0x0B.

Bits 24-31 toman el resto: 0x00.

El procesador ensamblará estas piezas internamente para saber dónde empieza el segmento.

Cambio en los Flags (De 0b11001111 a 0b01000000):

En el modelo plano, el primer bit del flag (la Granularidad) estaba en 1. Esto significa que el "Límite" de 0xFFFF se multiplica por bloques de 4 Kilobytes, dándote acceso a los 4 Gigabytes totales de RAM.

En el modelo segmentado, se cambia el flag de Granularidad a 0. Esto significa que el "Límite" de 0xFFFF (65535 en decimal) se cuenta byte por byte. Se le está indicando al procesador que el segmento de memoria de video mide exactamente 64 Kilobytes. Si el programa intenta escribir en la dirección 65536 o superior dentro de este segmento, tiene que lanzar un error general (Segmentation Fault) porque se salió de la pantalla.

6.3. Bits de Acceso

1. El Cambio en el Código: El Byte de Acceso

Para hacer que el Segmento de Datos sea de "Solo Lectura" (Read-Only), es necesario modificar un único bit en el Byte de Acceso del descriptor `gdt_data`.

En el código anterior, el byte de acceso era `0b10010010`.

El Bit 1 (contando de derecha a izquierda, empezando desde el 0) es el bit de Lectura/Escritura (R/W) para los segmentos de datos. Si está en 1, el segmento permite leer y escribir. Si se lo pone en 0, el segmento se vuelve de solo lectura.

6.4. Registros de Segmento

1) Selector de Segmento.

Un Selector de Segmento no es una dirección de memoria. Le indica a la Unidad de Gestión de Memoria (MMU) del procesador que busque en la tabla GDT el descriptor correspondiente, el cual contiene la dirección base física real, el tamaño límite y los permisos de acceso de ese segmento de memoria.

Un Selector es un número de 16 bits, y el hardware de Intel no lo lee como un número entero, sino que lo divide en tres partes (leyendo de derecha a izquierda):

Bits 0-1 (RPL - Requested Privilege Level): Definen el nivel de seguridad o "Anillo" (Ring) que solicita el programa. 00 es Ring 0 (Kernel, máximo poder) y 11 es Ring 3 (Usuario, sin privilegios).

Bit 2 (TI - Table Indicator): Le dice al procesador en qué tabla buscar. 0 significa buscar en la GDT (Tabla Global). 1 significa buscar en la LDT (Tabla Local).

Bits 3-15 (Index): Es el número de casillero real dentro de la tabla. Como cada descriptor en la GDT mide exactamente 8 bytes, este índice salta de 8 en 8.

2) Anatomía del valor `0x08` (El Segmento de Código)

En el código, se utilizó el salto lejano para cargar el registro CS (Code Segment) así: `ljmp $0x08, $modo_protegido`.

Si se convierte `0x08` (Hexadecimal) a Binario se obtiene: `0000 0000 0000 1000`.

Cortándolo según las reglas de Intel:

RPL (Bits 0-1): `00` -> Se quiere ejecutar este código con privilegios de Kernel (Ring 0).

TI (Bit 2): `0` -> Busca en la GDT.

Índice (Bits 3-15): `0000000000000001` -> Descriptor número 1.

¿Por qué el Descriptor 1? Recordando como se armó la tabla en la memoria:

`gdt_null` (Desplazamiento 0 bytes) -> Descriptor 0 (Ignorado por seguridad).

`gdt_code` (Desplazamiento 8 bytes) -> Descriptor 1. Acá está el código.

`gdt_data` (Desplazamiento 16 bytes) -> Descriptor 2.

Por lo tanto, cargar `0x08` en el registro CS es la forma matemática de decirle al procesador que las reglas para ejecutar instrucciones están en el primer descriptor válido de la GDT.

3) Anatomía del valor `0x10` (Los Segmentos de Datos)

Inmediatamente después del salto, se cargan los registros de datos (DS, ES, FS, GS, SS) utilizando la instrucción `mov` con el valor `$0x10`.

Si se convierte `0x10` (Hexadecimal) a Binario se obtiene: `0000 0000 0001 0000`.

Se cortan los bits nuevamente:

RPL (Bits 0-1): `00` -> Privilegios de Kernel (Ring 0).

TI (Bit 2): `0` -> Busca en la GDT.

Índice (Bits 3-15): `0000000000000010` -> Descriptor número 2.

Al cargar `0x10`, se apunta directamente al descriptor `gdt_data` (que estaba a 16 bytes del inicio, de ahí el número `0x10` en hexadecimal). A partir de ese momento, cualquier lectura o escritura en RAM o manejo de la Pila (Stack) utilizará los límites y permisos de datos que se definieron en ese descriptor.

7. Conclusiones

1) El fin del Modo Real y la estandarización de UEFI Class 3: La eliminación del Módulo de Soporte de Compatibilidad (CSM) en los equipos modernos marca el fin del arranque nativo en Modo Real (16 bits) a nivel de usuario. Actualmente, el firmware inicializa el hardware y transiciona a 32/64 bits.

2) La GDT como cimiento de la seguridad del sistema: Al reemplazar el direccionamiento físico directo por Selectores de Segmento, la Unidad de Gestión de Memoria (MMU) impone fronteras perimetrales y anillos de privilegios, garantizando que un error de segmentación no provoque el colapso total de la máquina.

3) El balance entre Abstracción y Control Absoluto: La evolución hacia UEFI y el uso de Linkers simplifican el desarrollo de sistemas, pero introducen cajas negras propietarias que pueden ser vulneradas (ej. BootHole). El estudio de Modo Protegido puro y alternativas como Coreboot reafirman la necesidad de comprender y poder auditar los procesos de inicialización más fundamentales del hardware.

8. Anexo

Enlace al Repositorio:

<https://github.com/GastonYomaha/SistemasDeComputacion/tree/main/TP3>